Name – Bhakare Purva Shivaji

Roll no – 08

Practical no – 06

```
# 1. IMPORT LIBRARIES
import os
import zipfile
import glob

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# 2. ZIP FILE PATH
zip_path = r"C:\Users\PVGIT\Downloads\archive (12).zip"
extract_folder = r"C:\Users\PVGIT\Downloads\house_price_dataset"

# 3. EXTRACT ZIP FILE
print("Extracting dataset...")
os.makedirs(extract_folder, exist_ok=True)
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_folder)
print("Dataset extracted successfully!")

# 4. FIND CSV FILE
```

```
csv_files = glob.glob(
    os.path.join(extract_folder, "**", "*.csv"),
    recursive=True
)
if len(csv_files) == 0:
    raise FileNotFoundError(
        "No CSV file found inside the ZIP file."
    )
print("\nCSV files found:")
for file in csv_files:
    print(file)
# Use first CSV file
csv_path = csv_files[0]
print("\nUsing file:")
print(csv_path)

# 5. LOAD DATASET
df = pd.read_csv(csv_path)
print("\nFirst 5 rows:")
print(df.head())
print("\nDataset shape:")
print(df.shape)
print("\nColumns:")
print(df.columns.tolist()

# 6. HANDLE MISSING VALUES
print("\nMissing values:")
print(df.isnull().sum())
df = df.dropna()

# 7. FIND TARGET COLUMN
```

```
possible_targets = [
    "price",
    "Price",
    "PRICE",
    "SalePrice",
    "sale_price",
    "house_price",
    "HousePrice"
]
target_column = None
for column in possible_targets:
    if column in df.columns:
        target_column = column
        break
if target_column is None:
    numerical_columns = df.select_dtypes(
        include=[np.number]
    ).columns
    if len(numerical_columns) == 0:
        raise ValueError(
            "No numerical target column found."
        )
    target_column = numerical_columns[-1]
print("\nTarget column:")
print(target_column)

# 8. SELECT FEATURES
X = df.drop(columns=[target_column])
y = df[target_column]
# Select only numerical features
X = X.select_dtypes(include=[np.number])
```

```
if X.shape[1] == 0:
    raise ValueError(
        "No numerical features available for regression."
    )
print("\nFeatures used:")
print(X.columns.tolist())

# 9. TRAIN-TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
# 10. LINEAR REGRESSION
linear_model = Pipeline([
    ("scaler", StandardScaler()),
    ("regressor", LinearRegression())
])
linear_model.fit(X_train, y_train)
linear_predictions = linear_model.predict(X_test)
# 11. LINEAR REGRESSION METRICS
linear_mse = mean_squared_error(
    y_test,
    linear_predictions
)
linear_mae = mean_absolute_error(
    y_test,
    linear_predictions
```

```
)
linear_r2 = r2_score(
    y_test,
    linear_predictions
)
# 12. POLYNOMIAL REGRESSION
polynomial_model = Pipeline([
    ("scaler", StandardScaler()),
    ("polynomial",
     PolynomialFeatures(
         degree=2,
         include_bias=False
     )),
    ("regressor", LinearRegression())
])
polynomial_model.fit(X_train, y_train)
polynomial_predictions = polynomial_model.predict(X_test)
# 13. POLYNOMIAL REGRESSION METRICS
polynomial_mse = mean_squared_error(
    y_test,
    polynomial_predictions
)
polynomial_mae = mean_absolute_error(
    y_test,
    polynomial_predictions
)
polynomial_r2 = r2_score(
    y_test,
    polynomial_predictions
)
# 14. PRINT RESULTS
```

```
print("\n")
print("=" * 70)
print("            LINEAR REGRESSION RESULTS")
print("=" * 70)
print("MSE      :", linear_mse)
print("MAE      :", linear_mae)
print("R2 Score :", linear_r2)
print("\n")
print("=" * 70)
print("          POLYNOMIAL REGRESSION RESULTS")
print("=" * 70)
print("MSE      :", polynomial_mse)
print("MAE      :", polynomial_mae)
print("R2 Score :", polynomial_r2)

# 15. MODEL COMPARISON TABLE
results = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Polynomial Regression"
    ],
    "MSE": [
        linear_mse,
        polynomial_mse
    ],
    "MAE": [
        linear_mae,
        polynomial_mae
    ],
    "R2 Score": [
        linear_r2,
```

```
        polynomial_r2
    ]
})

print("\n")
print("=" * 70)
print("                    MODEL COMPARISON")
print("=" * 70)
print(results.to_string(index=False))
# GRAPH 1
# ACTUAL VS PREDICTED - LINEAR REGRESSION
plt.figure(figsize=(8, 6))
plt.scatter(
    y_test,
    linear_predictions,
    color="blue",
    alpha=0.6
)
# Perfect prediction line
minimum = min(y_test.min(), linear_predictions.min())
maximum = max(y_test.max(), linear_predictions.max())
plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    color="red",
    linewidth=2
)
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title(
    "Linear Regression: Actual vs Predicted Prices"
```

```
)
plt.grid(True)

plt.tight_layout()
plt.show()
# GRAPH 2
# ACTUAL VS PREDICTED - POLYNOMIAL REGRESSION
plt.figure(figsize=(8, 6))
plt.scatter(
    y_test,
    polynomial_predictions,
    color="green",
    alpha=0.6
)
minimum = min(
    y_test.min(),
    polynomial_predictions.min()
)
maximum = max(
    y_test.max(),
    polynomial_predictions.max()
)
plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    color="red",
    linewidth=2
)
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title(
```

```
    "Polynomial Regression: Actual vs Predicted Prices"
)

plt.grid(True)
plt.tight_layout()
plt.show()
# GRAPH 3
# MODEL METRICS COMPARISON
metrics = ["MSE", "MAE", "R2 Score"]
linear_values = [
    linear_mse,
    linear_mae,
    linear_r2
]
polynomial_values = [
    polynomial_mse,
    polynomial_mae,
    polynomial_r2
]
x = np.arange(len(metrics))
width = 0.35
plt.figure(figsize=(9, 6))
plt.bar(
    x - width / 2,
    linear_values,
    width,
    label="Linear Regression",
    color="blue"
)
plt.bar(
    x + width / 2,
```

```
    polynomial_values,
    width,
    label="Polynomial Regression",
    color="green"
)
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.title(
    "Linear Regression vs Polynomial Regression"
)
plt.xticks(x, metrics)
plt.legend()
plt.grid(
    axis="y",
    alpha=0.3
)
plt.tight_layout()
plt.show()
# GRAPH 4
# ACTUAL PRICE VS FIRST FEATURE
first_feature = X.columns[0]
plt.figure(figsize=(8, 6))
plt.scatter(
    X_test[first_feature],
    y_test,
    color="purple",
    alpha=0.6
)
plt.xlabel(first_feature)
plt.ylabel("House Price")
plt.title(
```

```
    "House Price vs " + first_feature
)

plt.grid(True)
plt.tight_layout()
plt.show()
# GRAPH 5
# SAMPLE PREDICTIONS
number_of_samples = min(20, len(y_test))
actual_values = y_test.values[:number_of_samples]
linear_values_sample = (
    linear_predictions[:number_of_samples]
)
polynomial_values_sample = (
    polynomial_predictions[:number_of_samples]
)
indices = np.arange(number_of_samples)
plt.figure(figsize=(12, 6))
plt.plot(
    indices,
    actual_values,
    marker="o",
    label="Actual Price",
    color="black"
)
plt.plot(
    indices,
    linear_values_sample,
    marker="x",
    label="Linear Prediction",
    color="blue"
```

```
)
plt.plot(
    indices,
    polynomial_values_sample,
    marker="s",
    label="Polynomial Prediction",
    color="green"
)
plt.xlabel("House Sample")
plt.ylabel("Price")
plt.title(
    "Actual vs Predicted House Prices"
)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
# FINAL CONCLUSION
print("\n")
print("=" * 70)
print("                    CONCLUSION")
print("=" * 70)
print("\nLinear Regression:")
print("MSE      =", linear_mse)
print("MAE      =", linear_mae)
print("R2 Score =", linear_r2)
print("\nPolynomial Regression:")
print("MSE      =", polynomial_mse)
print("MAE      =", polynomial_mae)
print("R2 Score =", polynomial_r2)
print("\nMetric interpretation:")
```

```
print("Lower MSE  = Better")
print("Lower MAE  = Better")
print("Higher R2  = Better")

print("\nProgram completed successfully!")
```

OUTPUT-

```
Extracting dataset...
Dataset extracted successfully!

CSV files found:
C:\Users\PVGIT\Downloads\house_price_dataset\House Price Prediction Dataset.csv

Using file:
C:\Users\PVGIT\Downloads\house_price_dataset\House Price Prediction Dataset.csv

First 5 rows:
   Id  Area  Bedrooms  Bathrooms  Floors  YearBuilt  Location  Condition  \
0   1  1360         5          4       3       1970  Downtown  Excellent
1   2  4272         5          4       3       1958  Downtown  Excellent
2   3  3592         2          2       3       1938  Downtown       Good
3   4   966         4          2       2       1902  Suburban       Fair
4   5  4926         1          4       2       1975  Downtown       Fair

  Garage   Price
0     No  149919
1     No  424998
2     No  266746
3    Yes  244020
4    Yes  636056

Dataset shape:
```

(2000, 10)

Columns:

['Id', 'Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt', 'Location', 'Condition', 'Garage', 'Price']

Missing values:

Id 0

Area 0

Bedrooms 0

Bathrooms 0

Floors 0

YearBuilt 0

Location 0

Condition 0

Garage 0

Price 0

dtype: int64

Target column:

Price

Features used:

['Id', 'Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt']

Training samples: 1600

Testing samples: 400

==========

LINEAR REGRESSION RESULTS

==========

MSE : 78724363458.53015

MAE : 244166.1670510628

R2 Score : -0.011896515104903527

==========

POLYNOMIAL REGRESSION RESULTS

==========

MSE : 79907326011.10791

MAE : 246346.2724844325

R2 Score : -0.027101918259209157

==========

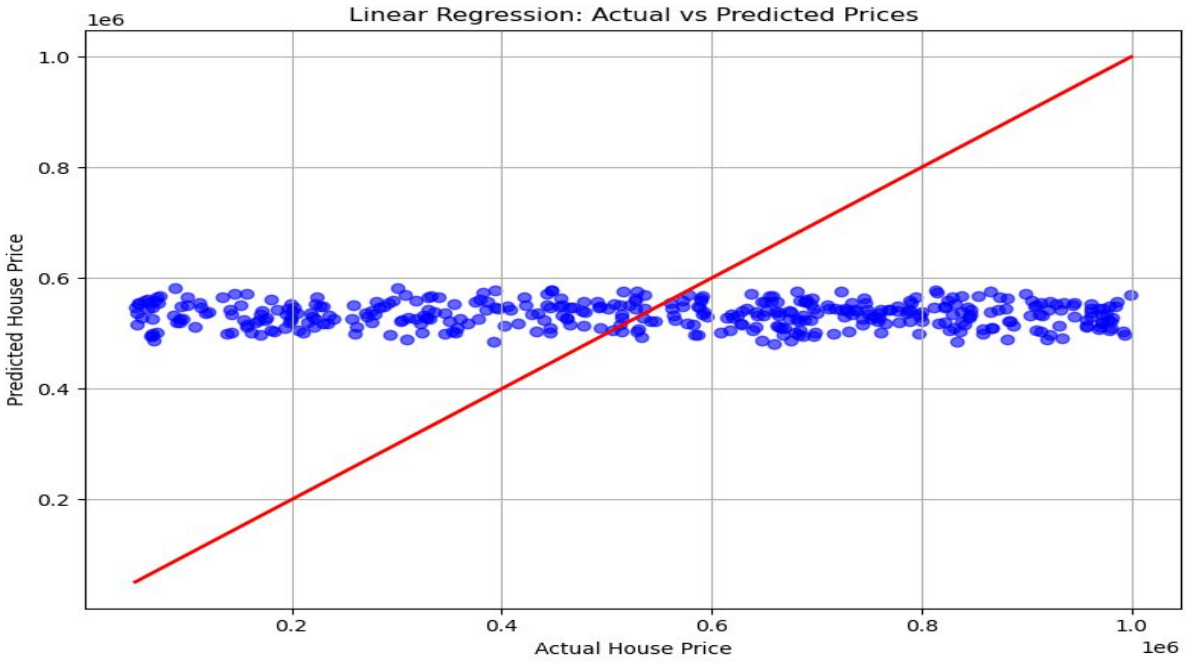
MODEL COMPARISON

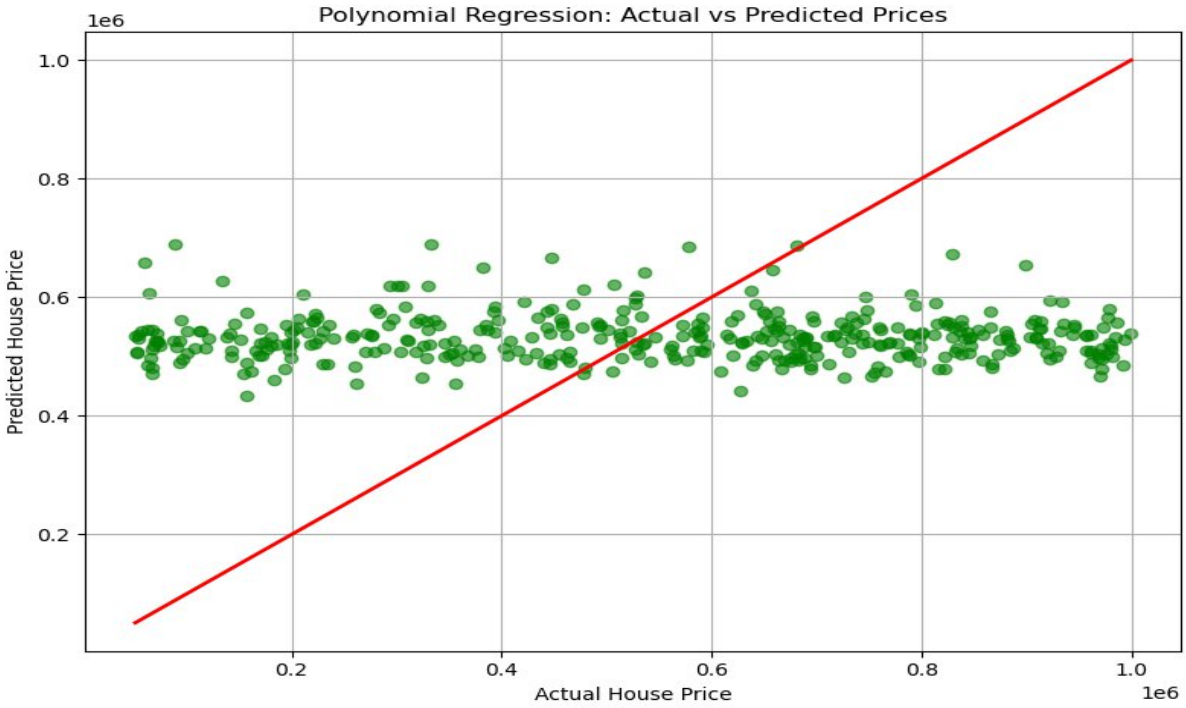
==========

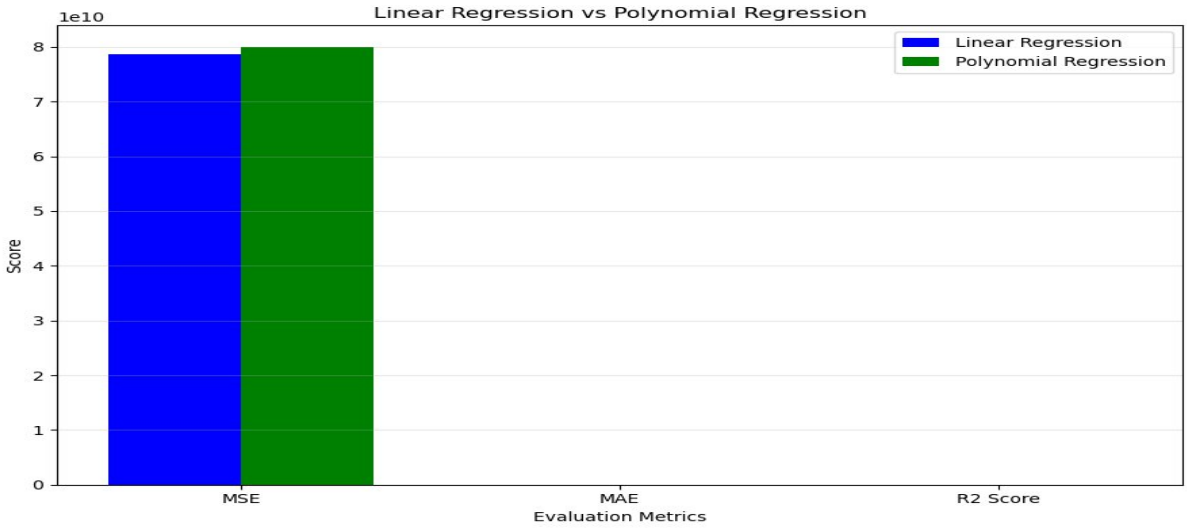
Model MSE MAE R2 Score

Linear Regression 7.872436e+10 244166.167051 -0.011897

Polynomial Regression 7.990733e+10 246346.272484 -0.027102




Polynomial Regression: Actual vs Predicted Prices
1e6
1.0
0.8
0.6
0.4
0.2
Predicted House Price
0.2
0.4
0.6
0.8
1.0
1e6
Actual House Price


Linear Regression vs Polynomial Regression
1e10
8
7
6
5
4
3
2
1
0
Score
Linear Regression
Polynomial Regression
MSE
MAE
R2 Score
Evaluation Metrics





==========

CONCLUSION

==========

Linear Regression:

MSE = 78724363458.53015

MAE = 244166.1670510628

R2 Score = -0.011896515104903527

Polynomial Regression:

MSE = 79907326011.10791

MAE = 246346.2724844325

R2 Score = -0.027101918259209157

Metric interpretation:

Lower MSE = Better

Lower MAE = Better

Higher R2 = Better

Program completed successfully!